

AI Assisted Coding

Assignment –1.2

Ch. Nithin Reddy || 2303A51550 || Batch:- 8

Task 1: AI-Generated Logic Without Modularization (Factorial without Functions)

- Scenario

You are building a small command-line utility for a startup intern onboarding task. The program is simple and must be written quickly without modular design.

- Task Description

Use GitHub Copilot to generate a Python program that computes a mathematical product-based value (factorial-like logic) directly in the main execution flow, without using any user-defined functions.

- Constraint:

- Do not define any custom function
- Logic must be implemented using loops and variables only

- Expected Deliverables

- A working Python program generated with Copilot assistance
- Screenshot(s) showing:
- The prompt you typed
- Copilot's suggestions
- Sample input/output screenshots
- Brief reflection (5–6 lines):
- How helpful was Copilot for a beginner?
- Did it follow best practices automatically?

```
lab1_2.py > ...
1  # Write a Python program to calculate factorial of a number using loops only.
2
3  # Constraints:
4  # - Do not use any user-defined functions
5  # - Implement logic in main program
6  # - Take input from user
7  # - Use loop and variables only
8  # - Display result clearly
9  # Get input from user
10 number = int(input("Enter a number to calculate its factorial: "))
11 # Initialize factorial variable
12 factorial = 1
13 # Check if the number is negative, zero or positive
14 if number < 0:
15     print("Factorial is not defined for negative numbers.")
16 elif number == 0:
17     print("The factorial of 0 is 1.")
18 else:
19     # Calculate factorial using a loop
20     for i in range(1, number + 1):
21         factorial *= i
22     # Display the result
23     print(f"The factorial of {number} is {factorial}.")
24
25
26
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
SQL: SELECT
(1, 'Alice Johnson', 3)
(2, 'Bob Smith', 2) ...
● PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python39-64/Python.exe" "c:/Users/NITHIN REDDY/OneDrive/Desktop/All courses/AI assistant/lab1_2.py"
Enter a number to calculate its factorial: 15
The factorial of 15 is 1307674368000.
○ PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> |
```

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

❖ Scenario

Your team lead asks you to review AI-generated code before committing it to a shared repository.

❖ Task Description

Analyze the code generated in Task 1 and use Copilot again to:

- Reduce unnecessary variables
- Improve loop clarity

➤ Enhance readability and efficiency

Hint:

Prompt Copilot with phrases like

“optimize this code”, “simplify logic”, or “make it more readable”

❖ Expected Deliverables

➤ Original AI-generated code

➤ Optimized version of the same code

➤ Side-by-side comparison

➤ Written explanation:

- What was improved?

- Why the new version is better (readability, performance, maintainability).

```
24
25 # Optimize this Python code by simplifying logic, improving readability,
26 # and reducing unnecessary variables. Keep it function-free and efficient.
27 number = int(input("Enter a number to calculate its factorial: "))
28 factorial = 1
29 if number < 0:
30     print("Factorial is not defined for negative numbers.")
31 elif number == 0:
32     print("The factorial of 0 is 1.")
33 else:
34     for i in range(1, number + 1):
35         factorial *= i
36     print(f"The factorial of {number} is {factorial}.")
37
38
39
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/OneDrive/Desktop/All courses/AI assistant/lab1_2.py"
Enter a number to calculate its factorial: 10
The factorial of 10 is 3628800.
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> |
```

Task 3: Modular Design Using AI Assistance (Factorial with Functions)

❖ Scenario

The same logic now needs to be reused in multiple scripts.

❖ Task Description

Use GitHub Copilot to generate a modular version of the program by:

- Creating a user-defined function
- Calling the function from the main block

❖ Constraints

- Use meaningful function and variable names
- Include inline comments (preferably suggested by Copilot)

❖ Expected Deliverables

- AI-assisted function-based program
- Screenshots showing:
 - o Prompt evolution
 - o Copilot-generated function logic
- Sample inputs/outputs
- Short note:
 - o How modularity improves reusability.

```

38 # Write a Python program to calculate factorial using a user-defined function.
39
40 # Requirements:
41 # - Create a function with meaningful name
42 # - Use loop logic inside the function
43 # - Add inline comments
44 # - Call the function from main block
45 # - Handle negative input
46 # - Display result clearly
47
48 def calculate_factorial(n):
49     # Initialize factorial variable
50     factorial = 1
51     # Check if the number is negative
52     if n < 0:
53         print("Factorial is not defined for negative numbers.")
54     # Check if the number is zero
55     elif n == 0:
56         print("The factorial of 0 is 1.")
57     else:
58         # Calculate factorial using a loop
59         for i in range(1, n + 1):
60             factorial *= i
61         print(f"The factorial of {n} is {factorial}.")
62
63 # Call the function from main block
64 number = int(input("Enter a number to calculate its factorial: "))
65 calculate_factorial(number)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```

PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY
n.exe" "c:/Users/NITHIN REDDY/OneDrive/Desktop/All courses/AI assistant/lab1_2.py"
Enter a number to calculate its factorial: 10
The factorial of 10 is 3628800.
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>

```

Task 4: Comparative Analysis – Procedural vs Modular AI Code (With vs Without Functions)

❖ Scenario

As part of a code review meeting, you are asked to justify design choices.

❖ Task Description

Compare the non-function and function-based Copilot-generated programs on the following criteria:

- Logic clarity
- Reusability

- Debugging ease
- Suitability for large projects
- AI dependency risk

❖ Expected Deliverables

Choose one:

- A comparison table

OR

- A short technical report (300–400 words).

```

OneDrive/Desktop/All courses/AI assistant/lab1_2.py
Comparison of Procedural and Modular Python Programs

```

Criterion	Procedural Program (Without Functions)	Modular Program (With Functions)
Logic clarity	Logic stays in one block, so small programs are easy to read at first. As code grows, the flow becomes crowded and harder to trace.	Logic is split into named functions, making each step easier to understand and follow.
Reusability	Code is usually written once and repeated manually when needed again.	Functions can be reused in many places without rewriting the same logic.
Debugging ease	Errors can be harder to isolate because everything is mixed in one section.	Bugs are easier to locate because each function can be tested separately.
Suitability for large projects	Works for short scripts or beginner-level tasks, but becomes difficult to manage in bigger projects.	Better for medium and large projects because work can be organized into smaller parts.
AI dependency risk	Higher risk of overreliance on AI for fixing or extending messy code when it becomes large.	Lower risk because structured code is easier for humans to review, improve, and maintain.

Short Explanation:

Procedural and modular programming both solve problems correctly, but they differ a lot in how the solution is organized. A procedural program without functions keeps all instructions in one continuous flow. This style is simple for very small tasks because the programmer can see the entire logic in one place. It is often useful for beginners when learning input, loops, and basic conditions. However, as the number of steps increases, the code becomes longer and less clear. When everything is written in one block, repeating logic is common, and even small changes may require editing several sections of the same program.

A modular program uses functions to divide the solution into smaller, meaningful units. Each function performs one specific task, such as taking input, processing data, or displaying output. This improves readability because the program can be understood step by step instead of line by line. Modular code is also more reusable. If the same logic is needed again, the function can be called instead of rewritten. This saves time and reduces mistakes.

Debugging is another major difference. In procedural code, an error may be hidden anywhere in a long block of statements, so finding the exact cause can take more effort. In modular code, the problem can often be narrowed down to a single function, making testing and correction easier. That is why modular programming is generally preferred for larger projects, where multiple features must work together in a clear and maintainable way.

AI dependency risk is also lower in modular programs. When code is well structured, a programmer can understand and verify it independently, even when AI tools are used for support. In contrast, unstructured procedural code may encourage more dependence on AI to explain, debug, or reorganize the logic. Overall, procedural programming is suitable for small and simple tasks, while modular programming is more effective for scalable, reusable, and maintainable software development.

```

PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>

```

Task 5: AI-Generated Iterative vs Recursive Thinking

❖ Scenario

Your mentor wants to test how well AI understands different computational paradigms.

❖ Task Description

Prompt Copilot to generate:

An iterative version of the logic

A recursive version of the same logic

❖ Constraints

Both implementations must produce identical outputs

Students must not manually write the code first

❖ Expected Deliverables

lab1_2.py > ...

```
67 # Generate a Python program that calculates factorial in two ways:
68 # 1. Iterative approach (using loop)
69 # 2. Recursive approach (using function)
70 # Requirements:
71 # - Both should produce identical outputs
72 # - Use clear variable and function names
73 # - Add comments explaining each approach
74 # - Take user input
75 # - Display both results for comparison
76 # - Handle negative inputs
77
78 # Output complete code in a single file.
79
80 # Get input from user
81 number = int(input("Enter a number to calculate its factorial: "))
82 # Iterative approach to calculate factorial
83 factorial_iterative = 1
84 if number < 0:
85     print("Factorial is not defined for negative numbers.")
86 elif number == 0:
87     print("The factorial of 0 is 1.")
88 else:
89     for i in range(1, number + 1):
90         factorial_iterative *= i
91     print(f"Iterative approach: The factorial of {number} is {factorial_iterative}.")
92 # Recursive approach to calculate factorial
93 def factorial_recursive(n):
94     if n < 0:
95         return None # Factorial is not defined for negative numbers
96     elif n == 0:
97         return 1 # The factorial of 0 is 1
98     else:
99         return n * factorial_recursive(n - 1) # Recursive call
100 factorial_recursive_result = factorial_recursive(number)
101 if factorial_recursive_result is None:
102     print("Recursive approach: Factorial is not defined for negative numbers.")
103 elif factorial_recursive_result == 1 and number == 0:
104     print("Recursive approach: The factorial of 0 is 1.")
105 else:
106     print(f"Recursive approach: The factorial of {number} is {factorial_recursive_result}.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE COMMENTS

neDrive/Desktop/All courses/AI assistant/lab1_2.py

Enter a number to calculate its factorial: 10

Iterative approach: The factorial of 10 is 3628800.

Recursive approach: The factorial of 10 is 3628800.

PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> █